

Follow-up on BOSS Codes for LPD Acoustic Communications

Anthony Jeongseok Lee

This repository consists of the code used for my original capstone project about block orthogonal sparse superposition (BOSS) codes and my independent follow-up after the project was over.

BOSS codes are generated using a dictionary matrix whose column vectors are linearly combined. In the original capstone project which I did with two teammates, we created a dictionary containing two sub-matrices that were constructed from wave and wind audio frames, so that the column vectors of each matrix could serve as components of codewords AND waveforms of ambient sound for LPD communications.

After that I wanted to see if the process could be simplified without compromising block error and detection rates, and also improve the source code to be a bit more intuitive for future uses. I found out my simplified version works as well as the encoder if both encoders are post-processed via QR decomposition for lower BLER. Without QR processing, the original encoder both shows lower BLER and probability of detection compared to the simplified version.

How BOSS Codes Work

Block orthogonal sparse superposition (BOSS) codes were introduced by Han et al. for ultra-reliable low latency (URLLC) communications. [1][2]

Encoding

BOSS codes combine encoding and modulation by using the bits to combine column vectors of symbols from a selected square sub-matrix, and the bit budget is determined by the information required to decide what vectors to decide from which sub-matrix and how to combine them.

The superposition of column vectors of a selected sub-matrix can be expressed as

$$\mathbf{c} = \mathbf{Ax} = [\mathbf{U}_1, \dots, \mathbf{U}_g, \dots, \mathbf{U}_G][\mathbf{0}_M^T, \dots, \mathbf{x}_g^T, \dots, \mathbf{0}_M^T]^T = \mathbf{U}_g \mathbf{x}_g,$$

Where $\mathbf{A} \in \mathbb{R}^{M \times (GM)}$ is called the **dictionary**, $\mathbf{x} \in \mathbb{R}^{(GM)}$ is the **message vector** with a non-zero **sub-message vector** $\mathbf{x}_g \in \mathbb{R}^M$, and each $\mathbf{U}_g$ is a **sub-matrix** that ideally should be unitary for optimal decoding. The segment $\mathbf{x}_g$, which has the role of picking column vectors to combine from the chosen unitary matrix, has $K = \sum_{l=1}^L K_l$ non-zero coefficients.

BOSS encoders also add the concept of **layers** to the message vector ($\mathbf{x} = \sum_{l=1}^L \mathbf{x}^{(l)}$), which increases the bit budget and therefore the code rate. Each non-zero entry falls under a layer l and can have a value chosen from $\mathcal{A}_l = \{\alpha_{l,i}, \dots, \alpha_{l,J_l}\}$, the set of possible coefficients for each l . Each layer has K_l elements, and the **candidate set** $\mathcal{M}^{(l)}$ of allowed potential non-zero indices for each layer is determined by the indices already chosen for previous layers, which means $|\mathcal{M}^{(l)}| = M - \sum_{i=1}^{l-1} K_i$.

When a bitstream is encoded, the first $\lfloor \log_2(G) \rfloor$ bits are used to pick the index g of the sub-matrix that will be used for encoding. For each layer, there are $\binom{\mathcal{M}^{(l)}}{K_l}$ possible combinations of indices for non-zero

coefficients of $\mathbf{x}_g$, and $|\mathcal{A}_l|^{K_l}$ possible K_l -tuple of values for those indices. So when the bits decide the sub-matrix and its corresponding message vector segment, a codeword of length M is generated via $\mathbf{c} = \mathbf{U}_g \mathbf{x}_g$. This process gives the encoder a code rate of

$$\frac{B}{M} = \frac{\lfloor \log_2(G) \rfloor + \sum_{l=1}^L (\lfloor \log_2 \left(\binom{\mathcal{M}^{(l)}}{K_l} \right) \rfloor + \lfloor \log_2 (|\mathcal{A}_l|^{K_l}) \rfloor)}{M}.$$

Decoding

For AWGN, the received signal $\mathbf{y} = \mathbf{c} + \mathbf{v}$ is decoded in two stages. In the first stage, for each hypothetical sub-matrix index g the signal is transformed using $\mathbf{y}_g = \mathbf{U}_g^T \mathbf{y}$, which is equal to $\mathbf{U}_g^{-1} \mathbf{y} = \mathbf{x}_g + \mathbf{v}_g$ if $\mathbf{U}_g$ is unitary. Then for each layer l , each index m within the received signal is given a log-APP score which measures the log probability that m was used as an index for a non-zero coefficient in the l^{th} layer:

$$\log P(x_{g,m}^{(l)} \in \mathcal{A}_l | y_{g,m}) = \log \frac{p(y_{g,m}, x_{g,m}^{(l)} \in \mathcal{A}_l)}{p(y_{g,m}, x_{g,m}^{(l)} \in \mathcal{A}_l) + p(y_{g,m}, x_{g,m}^{(l)} = 0)}$$

The terms in the numerator and denominator can be expanded to

$$p(y_{g,m} | x_{g,m}^{(l)} \in \mathcal{A}_l) P(x_{g,m}^{(l)} \in \mathcal{A}_l) = \frac{1}{J_l} \sum_{i=1}^{J_l} \frac{1}{\sqrt{2\pi\sigma_v^2}} e^{-\frac{|y_{g,m} - \alpha_{l,i}|^2}{2\sigma_v^2}} \frac{K_l}{|\mathcal{M}^{(l)}|},$$

and

$$p(y_{g,m} | x_{g,m}^{(l)} = 0) P(x_{g,m}^{(l)} = 0) = \frac{1}{\sqrt{2\pi\sigma_v^2}} e^{-\frac{|y_{g,m}|^2}{2\sigma_v^2}} \left(1 - \frac{K_l}{|\mathcal{M}^{(l)}|} \right).$$

Top K_l indices with the highest log-APP scores are assumed to be the indices used in $\mathbf{x}_g^{(l)}$, and the most likely value of $\mathbf{x}_g^{(l)}$ in each index is selected by picking the right values from $\mathcal{A}_l$ so that $\|\hat{\mathbf{x}}_g^{(l)} - \mathbf{y}_g\|_2$ is minimized.

Decoding performance can be increased even further by adding cyclic redundancy checks (CRC) in conjunction with MAP-list decoding. CRC is done by appending (degree) zeros to the original bitstream, long dividing it by the CRC polynomial to get the remainder which will be appended to the bitstream in the encoder, then dividing the appended bits and long dividing it by the polynomial to check if the remainder is 0. When the bits arrive at the decoder, a **list** of S most likely hypothesized estimates $\hat{\mathbf{x}}_g$ is chosen in the first stage for each g and get inverse mapped to their respective bitstreams. Then, only the estimated bits that pass the CRC check actually make it to the second stage.

Once the message vector segment is reconstructed as $\hat{\mathbf{x}}_g$ for each g in the first stage, the correct index is identified in the second stage by picking the index that minimizes $\|\mathbf{U}_g \hat{\mathbf{x}}_g - \mathbf{y}\|_2$.

The Simplified Encoder

Initialization

Not much has changed from the original implementation. Each sub-matrix for the encoder's dictionary is initialized using audio frames of length M , by mixing an orthogonal basis obtained via PCA, with spectral correction using the PSDs of its target ambient sound.

Let $\Psi = [\psi_1, \psi_2, \dots]$ be an $M \times N_{\text{fr}}$ matrix where each ψ_i is a time domain signal of M points, obtained from an audio file. Because BOSS codes are intended to operate in the short block length regime, M was chosen in the simulations as 256. To make each ψ_i sufficiently long and audible, the sampling rate was set to 1000Hz so that the codewords and frames would last for 0.25 seconds when played.

The audio frames aren't directly used as column vectors for the sub-matrix because no amount of orthogonality among the audio frames is guaranteed. So this issue is partially solved by extracting the orthonormal eigenvectors v_i of the correlation matrix $\frac{1}{N_{\text{fr}}} \Psi \Psi^T$, then replacing the magnitude response of each v_i with the square root of the average PSD of all N_{fr} frames, as shown below:

$$v_i^{\text{init}}[n] = \text{IFFT}(\sqrt{\overline{\text{PSD}}_{\Psi}[k]} e^{j\phi_i[k]})[n],$$

where $\overline{\text{PSD}}_{\Psi}[k] = \frac{1}{N_{\text{fr}}} \sum_{i=1}^{N_{\text{fr}}} |\text{FFT}(\psi_i)[k]|^2$, and $\phi_i[k] = \arg(\text{FFT}(v_i)[k])$.

This way, each $v_i^{\text{init}}[n]$, when inspected in the frequency domain, has a PSD that resembles the PSD of an audio frame, and the phase response $\phi[k]$ which sort of preserves the original structure of $\text{FFT}(v_i)[k]$. To make the column vectors represent real valued sound of a norm of 1, each $v_i^{\text{init}}[n]$ goes through another adjustment when being used as a column vector for a sub-matrix $\mathbf{U}_g^{\text{init}}$.

$$\mathbf{U}_g^{\text{init}} = \left[\frac{\text{Re}(v_1^{\text{init}})}{\|\text{Re}(v_1^{\text{init}})\|}, \dots, \frac{\text{Re}(v_M^{\text{init}})}{\|\text{Re}(v_M^{\text{init}})\|} \right]$$

Each $\mathbf{U}_g^{\text{init}}$ will be trained to balance between or improve orthogonality and covertness (similarity to audio frames) in the training stage that follows.

Training

My simplified version uses two loss terms for covertness and one term for orthogonality.

- $\mathcal{L}_{\text{psd}}$: MSE between the batch-averaged codeword PSD and average audio frame PSD, added so that codewords are trained to have similar frequency components to sound frames.

$$\mathcal{L}_{\text{psd}} = \frac{1}{M} \sum_{k=0}^{M-1} |\log(\text{PSD}(\mathbf{c})[k] + \epsilon) - \log(\overline{\text{PSD}}_{\Psi}[k] + \epsilon)|^2$$

```
c_g_psd = (torch.fft.fft(c_g, dim = -1).abs() ** 2).mean(dim=0) # using batch
averaged PSDs has more stable and predictable results
loss_psd += torch.mean ((torch.log(c_g_psd+ 1e-8)-torch.log(avg_psd[g]+ 1e-
8)).abs() **2)
```

- $\mathcal{L}_{\text{mel}}$: Same idea as above but with Mel spectrograms instead of PSDs. Added to account for human perception as well rather than just detection devices.

```
c_g_mel = compute_avg_mel_torch(c_g.T, SR, N_FFT, HOP_LENGTH, N_MELS)
loss_mel += torch.mean((torch.log(c_g_mel + 1e-8) - torch.log(avg_mel[g] + 1e-
8)).abs() **2)
```

- $\mathcal{L}_{\text{ortho}} = \frac{1}{M} \|\mathbf{U}^T \mathbf{U} - \mathbf{I}_M\|_F^2$

```
# average squared orthogonality error per column, the encoder.M factor also makes
the different losses more balanced
gram = A[g].T @ A[g]
loss_ortho += encoder.M*torch.mean(torch.abs((gram - I)) ** 2)
```

The capstone project's training process also included three other loss terms that involved STFT comparisons, a pretrained classifier, and the cross correlation between different sub-matrices. a Pareto curve of BLER and probability of detection (P_D) was also plotted to pick the best encoder. In the simplified follow-up, features were either removed or modified primarily to see how much I could get away with simplicity, and additionally for stable gradients over time.

Loss Term	Original Capstone Encoder	Simplified Encoder
PSD	MAE between a codeword's PSD and the average frame PSD, batch-averaging after comparison.	MSE between batch-averaged codeword PSD and the average frame PSD, modified for smoother and predictable gradient changes.
Mel	MAE between the Mel spectrograms of a codeword and random audio frames, batch-averaging after comparison.	MSE between the Mel spectrograms of batch averaged codewords and the average frame Mel spectrogram.
STFT	Comparison between the STFTs of two random codewords.	Removed for simplicity.
Classifier	Cross-entropy of a pre-trained sub-matrix classifier.	Removed for simplicity.
Inter-block	MSE of the entries of $\mathbf{U}_1^T \mathbf{U}_2$	Removed because this loss can't distinguish identical and distinct pairs in some cases. (e.g. $\mathbf{U}_1^T \mathbf{U}_2$ both have a squared Frobenius norm of M when $\mathbf{U}_1 = \mathbf{U}_2$ AND when $\mathbf{U}_2 = \mathbf{P} \mathbf{U}_1$ where $\mathbf{P}$ is a permutation matrix)
Orthogonality	MSE of the entries of $\mathbf{U}_g^T \mathbf{U}_g$ for each g	Another M is multiplied to the loss to represent average column-wise MSE.

After training, in the original capstone project, BLER and probability of detection (P_D) were measured for all intermediate dictionaries in each epoch and plotted to identify a Pareto front with the best encoder. In the follow-up this process was removed to reduce computation time.

QR Decomposition

A square matrix can be decomposed into a multiplication of two matrices $\mathbf{Q}$ and $\mathbf{R}$, where $\mathbf{Q}$ is an orthonormal basis obtained via a Gram-Schmidt process that starts with the first column vector, and $\mathbf{R}$ is an

upper triangular matrix. To compensate for a simpler training method I increased the number of epochs to make the sub-matrices closer to orthogonality, which should theoretically prevent too much waveform differences between $\mathbf{Q}$ and $\mathbf{U}$. In the simplified version of this project, QR decomposition is functionally necessary as the raw trained encoder has significantly higher BLER compared to its capstone counterpart.

Simulation Results

Parameters and Assumptions

5 encoders were compared for BLER and P_D

- `encoder_simplified_no_qr`
- `encoder_simplified_qr`
- `encoder_capstone_no_qr`
- `encoder_capstone_qr`
- `encoder_hadamard`: This encoder uses a Hadamard matrix and its column-permuted copies for the dictionary matrix.

The encoders in the simulation use the following parameters

- Codeword length: $M = 256$
- Number of sub-matrices: $G = 2$
- Number of layers: $L = 2$
- Non-zero coefficients per layer: $K_1 = K_2 = 1$
- Alphabet for each layer: $\mathcal{A}_1 = \{1\}$, $\mathcal{A}_2 = \{-1\}$
- CRC polynomial: 1011

As for the channel and decoder

- Channel type: AWGN
- E_b/N_0 was measured at integer dB values between -1 and 5dB.
- MAP-list size: $S = 4$
- CRC polynomial: 1011

The probability of detection P_D was defined as the probability that the L^2 distance between the normalized PSD of a received signal and the normalized average PSD of its corresponding sound frames passes a threshold. This threshold, while in the original capstone project was defined as the $\mu + 2\sigma$ value of the encoder's codeword PSD distribution, was modified to be dependent on the quantile $Q_{1-P_{FA}}$ derived from audio frames and a fixed false alarm rate P_{FA} .

$$P_D = \mathbb{P} \left(\left\| \frac{\text{PSD}(\mathbf{y})[k]}{\sum_{k=0}^{M-1} \text{PSD}(\mathbf{y})[k]} - \frac{\overline{\text{PSD}}_{\Psi}[k]}{\sum_{k=0}^{M-1} \overline{\text{PSD}}_{\Psi}[k]} \right\|_2 > \tau \right),$$

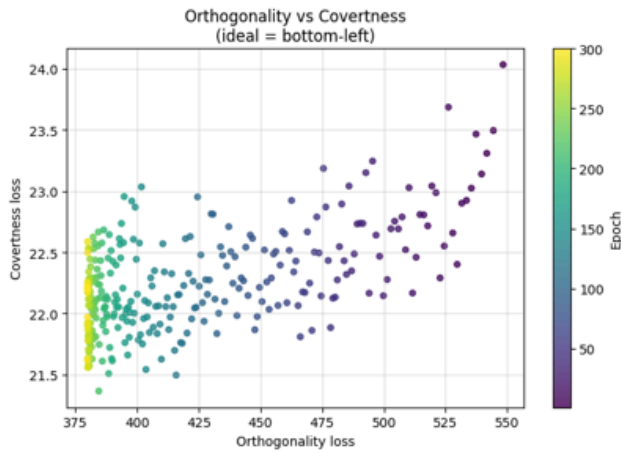
$$\tau = Q_{1-P_{FA}} \left(\left\{ \left\| \frac{\text{PSD}(\psi_i)[k]}{\sum_{k=0}^{M-1} \text{PSD}(\psi_i)[k]} - \frac{\overline{\text{PSD}}_{\Psi}[k]}{\sum_{k=0}^{M-1} \overline{\text{PSD}}_{\Psi}[k]} \right\|_2 \right\}_{i=1}^{N_{\text{fr}}} \right)$$

The idea of implementing a false-alarm-rate-based threshold was inspired by a review written by R. Diamant and L. Lampe. [3]

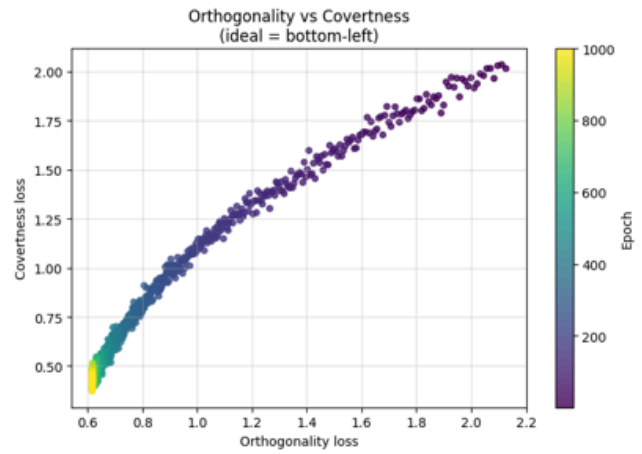
Ambient wave and wind recordings were obtained from Freesound. [4] [5]

Training Results

Training from capstone project

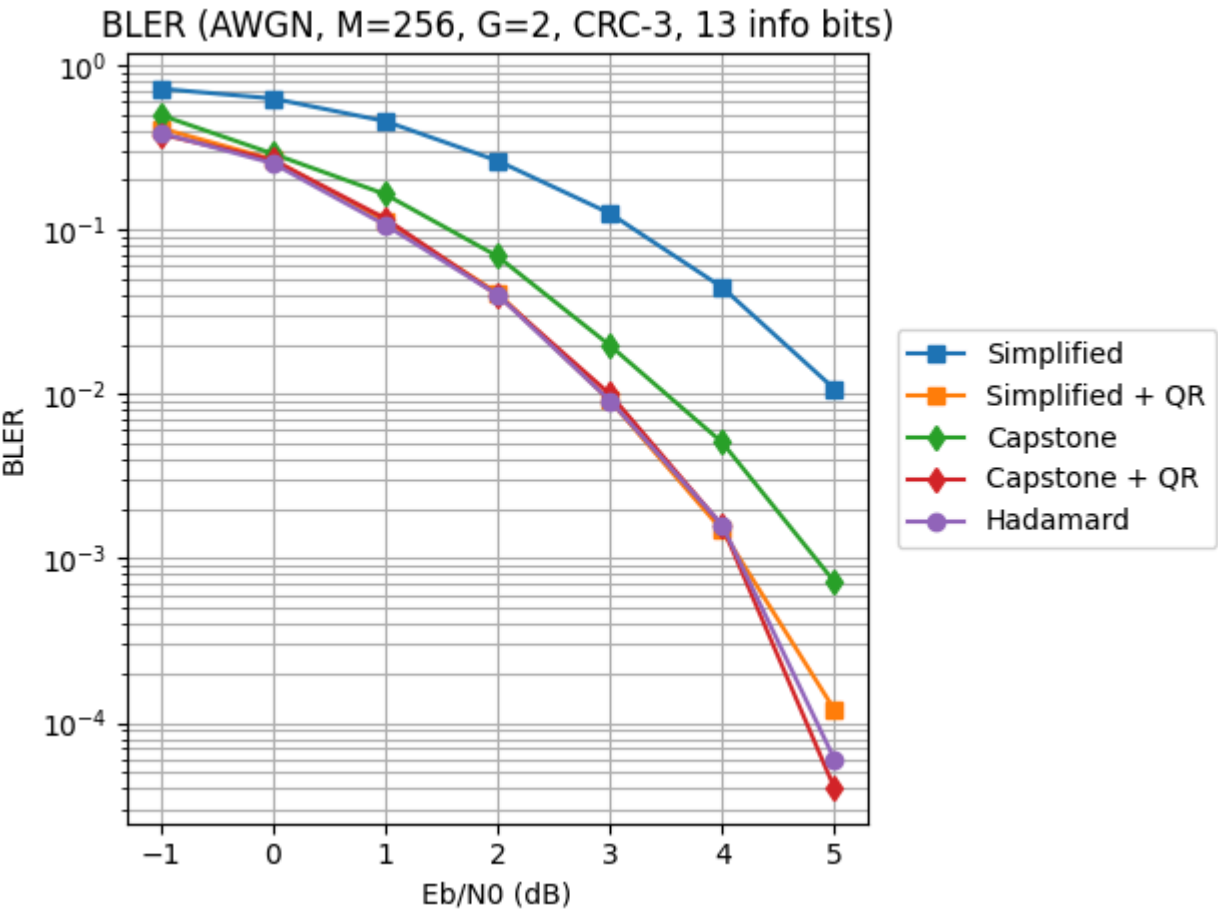


Simplified training



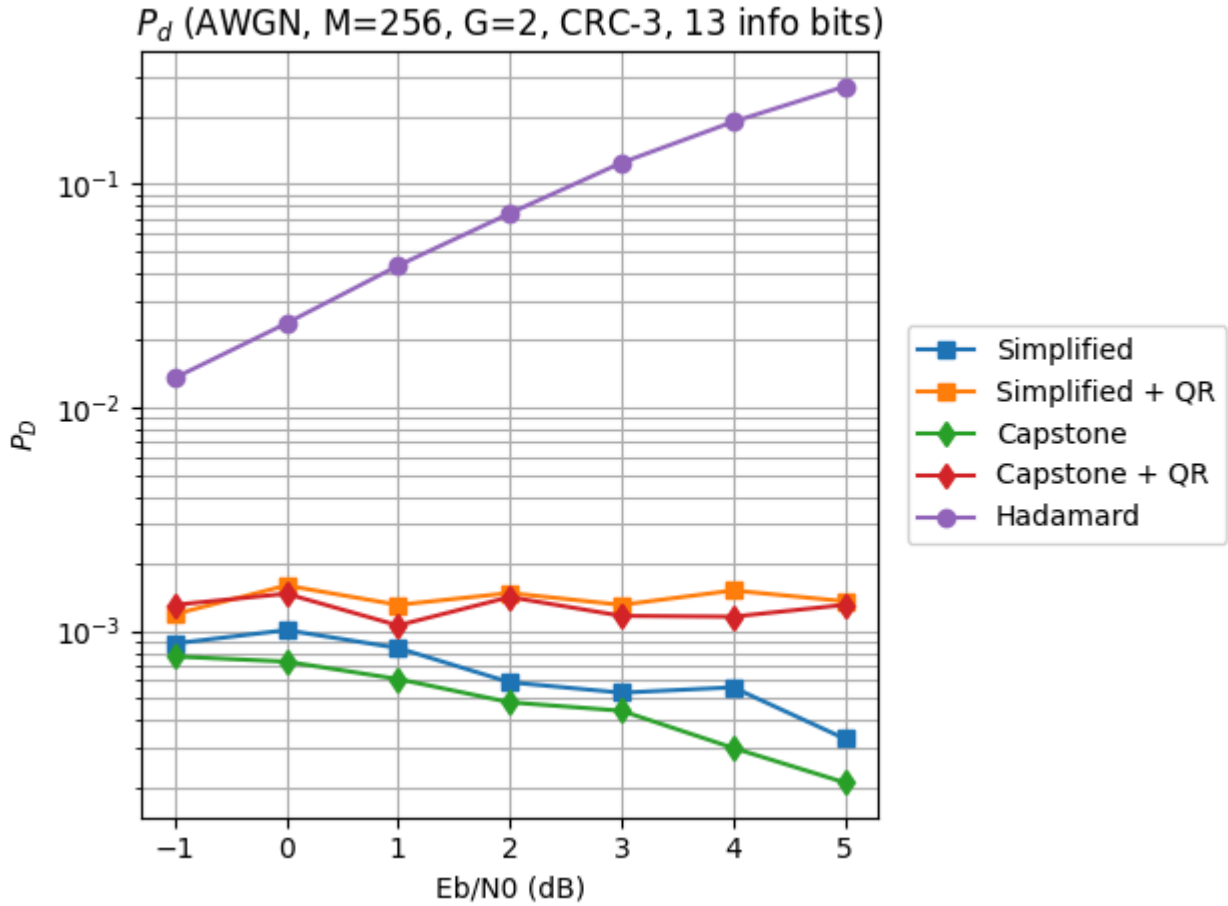
BLER

E_b/N_0 (dB)	Simplified (no QR)	Capstone (no QR)	Simplified (QR)	Capstone (QR)	Hadamard
-1	0.7180 (359/500)	0.4973 (373/750)	0.4133 (310/750)	0.3800 (285/750)	0.3867 (290/750)
0	0.6260 (313/500)	0.2890 (289/1000)	0.2670 (267/1000)	0.2650 (265/1000)	0.2530 (253/1000)
1	0.4560 (342/750)	0.1640 (287/1750)	0.1124 (281/2500)	0.1169 (263/2250)	0.1068 (267/2500)
2	0.2620 (262/1000)	0.0688 (258/3750)	0.0411 (257/6250)	0.0400 (260/6500)	0.0402 (261/6500)
3	0.1255 (251/2000)	0.0198 (252/12750)	0.0092 (252/27500)	0.0100 (250/25000)	0.0091 (251/27500)
4	0.0445 (256/5750)	0.0051 (250/49250)	0.0015 (74/50000)	0.0016 (79/50000)	0.0016 (80/50000)
5	0.0107 (252/23500)	0.0007 (36/50000)	0.0001 (6/50000)	0.0000 (2/50000)	0.0001 (3/50000)



Probability of Detection

E_b/N_0 (dB)	Simplified (no QR)	Capstone (no QR)	Simplified (QR)	Capstone (QR)	Hadamard
-1	0.0009 (88/100000)	0.0008 (77/100000)	0.0012 (119/100000)	0.0013 (131/100000)	0.0136 (1357/100000)
0	0.0010 (101/100000)	0.0007 (73/100000)	0.0016 (160/100000)	0.0015 (147/100000)	0.0238 (2383/100000)
1	0.0008 (84/100000)	0.0006 (61/100000)	0.0013 (131/100000)	0.0011 (106/100000)	0.0432 (4318/100000)
2	0.0006 (59/100000)	0.0005 (48/100000)	0.0015 (148/100000)	0.0014 (142/100000)	0.0740 (7404/100000)
3	0.0005 (53/100000)	0.0004 (44/100000)	0.0013 (131/100000)	0.0012 (117/100000)	0.1246 (12461/100000)
4	0.0006 (56/100000)	0.0003 (30/100000)	0.0015 (152/100000)	0.0012 (116/100000)	0.1901 (19012/100000)
5	0.0003 (33/100000)	0.0002 (21/100000)	0.0014 (136/100000)	0.0013 (131/100000)	0.2736 (27357/100000)



Conclusion

The encoder obtained from this simplified follow up shows comparable BLER and probability of detection to its capstone equivalent IF QR decomposition is applied afterward. Without this post processing, the capstone's encoder shows much better performance, which may be due to either better loss functions or best epoch selection from the BLER vs P_D Pareto front.

Improving the training method, a better way to accurately represent ambient sound with a small number of symbols, a better definition of P_D , and simulations with harsher channels would make this project more applicable to LPD communications.

References and Audio Sources

References

[1] D. Han, J. Park, Y. Lee, H. V. Poor, and N. Lee, "Block Orthogonal Sparse Superposition Codes for Ultra-Reliable Low-Latency Communications," *IEEE Trans. Commun.*, vol. 71, no. 12, pp. 6884–6897, Dec. 2023, doi: 10.1109/TCOMM.2023.3317912.

[2] D. Han, B. Lee, M. Jang, D. Lee, S. Myung, and N. Lee, "Block Orthogonal Sparse Superposition Codes for L^3 Communications: Low Error Rate, Low Latency, and Low Transmission Power," *IEEE J. Sel. Areas Commun.*, doi: 10.1109/JSAC.2025.3531569.

[3] R. Diamant and L. Lampe, "Low Probability of Detection for Underwater Acoustic Communication: A Review," *IEEE Access*, vol. 6, pp. 19099–19112, 2018, doi: 10.1109/ACCESS.2018.2818110.

Audio Sources

[4] YevgVerh, "Ocean_coast_04_092025_0659AM," Freesound, CC0. [Online]. Available: <https://freesound.org/people/YevgVerh/sounds/827530/>

[5] craigsmith, "Cold Day Wind," Freesound, CC0. [Online]. Available: <https://freesound.org/people/craigsmith/sounds/817182/>

Disclosure

- In contrast to the capstone, the follow-up is an independent project unaffiliated with any supervisor or institution.
- The source code was made with a mix of recycled code from the original capstone, generative AI and substantial human review and modification.